# 第二次作业报告

**23376251 陈家宝**

## 1. Objectives

This experiment aims to generate a 3D moons dataset for a classification task and compare the performance of different classification algorithms on this dataset.

（1）Generate a 3D dataset with 1,000 samples in total, divided into two classes: C0 and C1.

（2）Generate an additional 500 samples as the test set (250 per class), drawn from the same distribution as the training set.

（3）Compare the classification performance of the following models on the test set: Decision Tree, AdaBoost + Decision Tree (AdaBoost + DT), and Support Vector Machine (SVM) with three kernel functions (Linear, RBF, and Polynomial).

（4）Analyze the classification performance of each model and the underlying reasons, and visualize the 3D classification results alongside confusion matrices.

## 2. Data Generation

A custom function make_moons_3d was implemented to generate the three-dimensional moon-shaped dataset. The core idea extends the classic 2D make_moons by adding a nonlinear third dimension z = sin(2t), forming a volumetric crescent distribution. Gaussian noise is applied to increase data diversity.

Data generation function:

```
def make_moons_3d(n_samples=500, noise=0.2):
    t = np.linspace(0, 2 * np.pi, n_samples)
    x = 1.5 * np.cos(t)
    y = np.sin(t)
    z = np.sin(2 * t)
    X = np.vstack([
        np.column_stack([x, y, z]),
        np.column_stack([-x, y - 1, -z])
    ])
    labels = np.hstack([np.zeros(n_samples), np.ones(n_samples)])
    X += np.random.normal(scale=noise, size=X.shape)
    return X, labels
```

Key dataset properties:

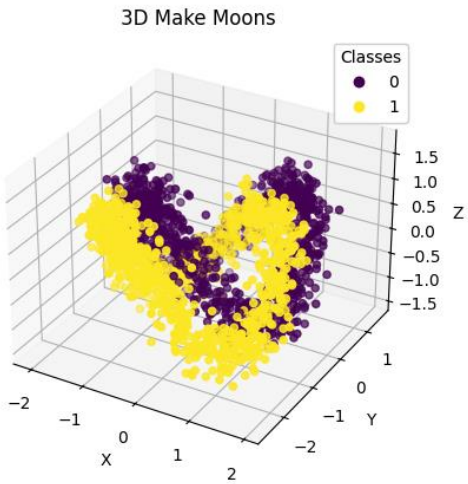
Training set: 1,000 samples (500 per class); Test set: 500 samples (250 per class)

Classes are balanced — class labels are C0 (label 0) and C1 (label 1)

The Z dimension introduces a sin(2t) nonlinear transformation, creating a 3D crescent-shaped distribution

Gaussian noise with standard deviation 0.2 is added to simulate real-world data uncertainty

**Figure 1: 3D visualization of the training set (two crescent classes)**




# 3. Models and Training

Five classifiers were selected for comparison, covering tree-based methods, ensemble learning, and SVMs with three kernel functions. Parameter settings are listed in the table below.

| Model | Parameter Settings |
|---|---|
| **Decision Tree** | max_depth=5 |
| **AdaBoost + DT** | Base estimator: DecisionTree(max_depth=1), n_estimators=50 |
| **SVM (Linear)** | kernel='linear' |
| **SVM (RBF)** | kernel='rbf' |
| **SVM (Poly)** | kernel='poly', degree=3 |

Training and evaluation procedure:

All models are fitted (trained) on the training set of 1,000 samples

The test set (500 samples) is used solely for performance evaluation and does not participate in training

Evaluation metrics: Accuracy and Confusion Matrix

# 4. Results

## 4.1 Classification Accuracy

The classification accuracy of each model on the test set is shown in the table below (best result highlighted in green):

| Model | Accuracy |
|---|---|
| Decision Tree | 0.898 |
| AdaBoost + DT | 0.736 |
| SVM (Linear) | 0.678 |
| **SVM (RBF)** | **0.980** |
| SVM (Poly) | 0.862 |

SVM with RBF kernel ranks first with an accuracy of 0.980, while Linear SVM performs worst at 0.678.

## 4.2 3D Classification Visualization

The figures below show the 3D classification predictions of each model on the test set (colors indicate predicted class labels):

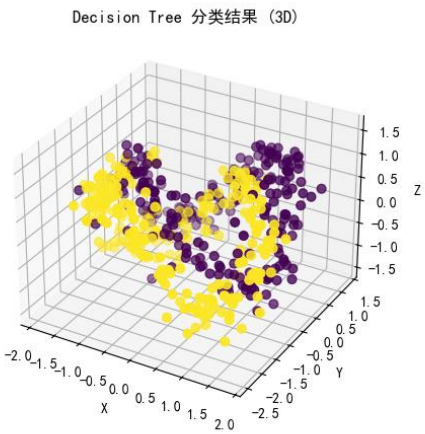



**Figure 2  Decision Tree — 3D Classification Result**

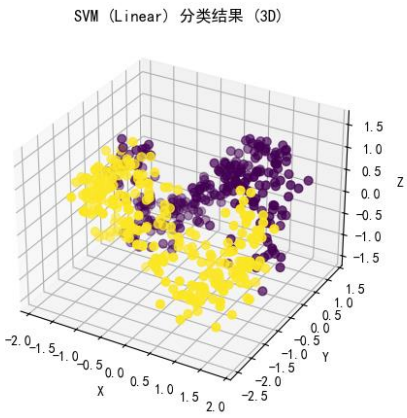



**Figure 3  AdaBoost + DT — 3D Classification Result**

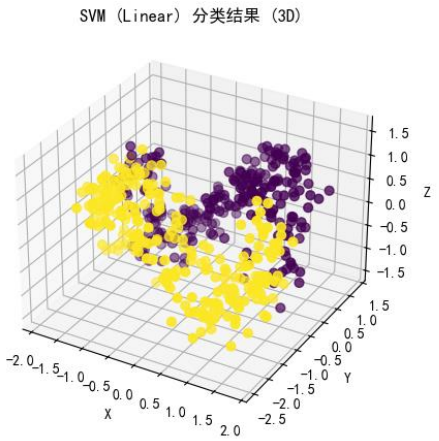



**Figure 4 SVM (Linear) — 3D Classification Result**

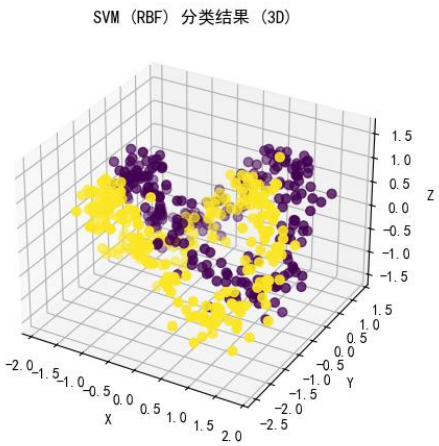



**Figure 5 SVM (RBF) — 3D Classification Result**

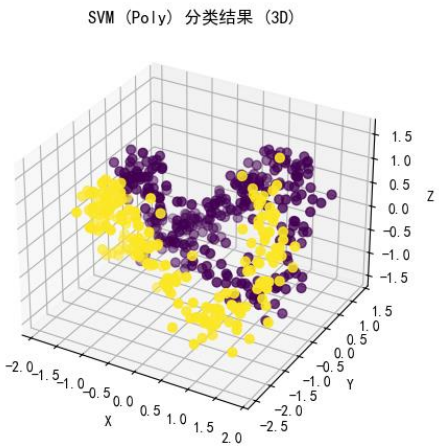



**Figure 6 SVM (Poly) — 3D Classification Result**

## 4.3 Confusion Matrix Analysis

A qualitative analysis of the confusion matrices for each model is provided below:

Decision Tree: approximately 222/250 correct for C0 and 227/250 for C1 — overall good performance

AdaBoost + DT: increased confusion between C0 and C1, indicating underfitting due to the shallow base estimator under noisy conditions

SVM (Linear): the linear kernel cannot fit the nonlinear crescent boundary, resulting in the most misclassifications

SVM (RBF): the nonlinear kernel achieves the best fit to the 3D crescent boundary, with very few misclassifications

SVM (Poly): the polynomial kernel captures some nonlinear structure, with error rate between Linear and RBF

Confusion matrix visualizations (see Figures 7 and 8):

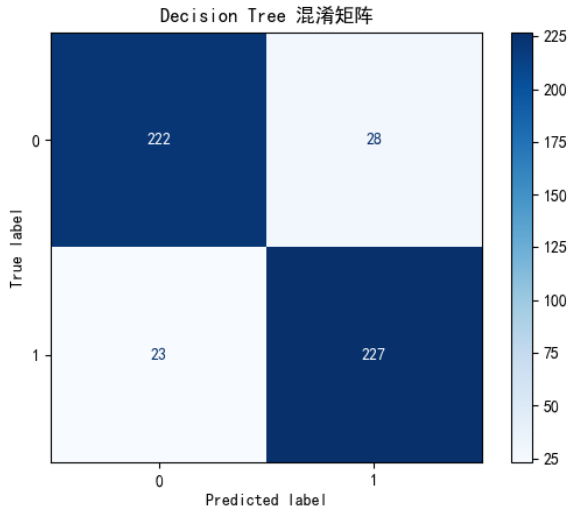



**Figure 7 Confusion matrices for selected models**

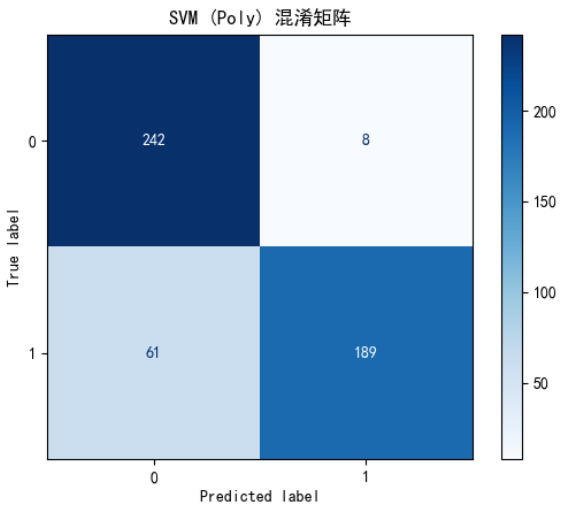



**Figure 8 Confusion matrices for selected models (continued)**

# 5. Analysis and Discussion

This section analyzes the performance differences among models from both mathematical principles and data characteristics.

## 5.1 Why SVM (RBF) Performs Best

The RBF (Radial Basis Function) kernel is defined as K(x, x') = exp(-gamma * ||x - x'||^2), which implicitly maps data into an infinite-dimensional feature space, enabling highly nonlinear decision boundaries. Since the 3D moon-shaped data is not linearly separable in the original feature space, the RBF kernel is well-suited to capture this complex curved boundary — resulting in the highest accuracy of 0.980 among all models.

## 5.2 Why Linear SVM Performs Worst

Linear kernel SVM seeks a hyperplane in the original feature space as the decision boundary. Since the 3D moons data is inherently nonlinearly separable, a linear hyperplane cannot effectively separate the two classes, leading to a large number of misclassifications and an accuracy of only 0.678 — close to random guessing.

## 5.3 Why AdaBoost + DT Underperforms

AdaBoost iteratively adjusts sample weights so that subsequent weak learners focus on previously misclassified samples. In this experiment, the base estimator (decision tree) has a maximum depth of 1 (a decision stump), which has very limited expressive power. On a noisy, nonlinearly structured 3D dataset, depth-1 stumps tend to underfit, and the ensemble effect actually falls behind a single Decision Tree of depth 5 — achieving only 0.736 accuracy.

## 5.4 Moderate Performance of Decision Tree and SVM (Poly)

A Decision Tree of depth 5 partitions the feature space using axis-aligned splits, which can partially approximate nonlinear boundaries, achieving a respectable accuracy of 0.898. The polynomial kernel SVM (degree=3) maps features into a polynomial space, capturing some nonlinear structure (accuracy 0.862), but its boundary flexibility is lower than that of the RBF kernel.

## 5.5Summary

In summary, model selection has a decisive impact on classification performance for nonlinear data:Kernel function choice is the key driver of SVM performance differences: RBF > Poly > Linear;The effectiveness of ensemble methods (AdaBoost) depends critically on the expressive power of the base estimator — overly weak estimators can degrade overall performance;For moderately complex nonlinear tasks, a single decision tree with appropriate depth offers a strong cost-performance trade-off

This experiment successfully built a 3D moon-shaped dataset and conducted a systematic comparison and analysis of five classification models. The main conclusions are as follows:

(1)A 3D Moons dataset with nonlinear structure was successfully generated (1,000 training samples and 500 test samples), with the data distribution verified through 3D visualization.

(2)SVM (RBF) achieved the best performance among all models (accuracy 0.980), demonstrating the strong capability of the RBF kernel to fit nonlinear data.

(3)Linear SVM achieved the lowest accuracy (0.678), confirming the limitations of linear

models on nonlinearly distributed data.

(4)AdaBoost failed to boost performance effectively when the base estimator was too weak (accuracy 0.736), performing worse than a single Decision Tree (0.898).

(5)For practical applications with nonlinearly distributed data, RBF kernel SVM or sufficiently deep tree models should be preferred, and visualization tools should be used to intuitively evaluate model performance.

# Appendix: Experiment Code

The complete Python code for this experiment is provided below:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay
import pandas as pd
import matplotlib

matplotlib.rcParams['font.sans-serif'] = ['SimHei']
matplotlib.rcParams['axes.unicode_minus'] = False

def make_moons_3d(n_samples=500, noise=0.1):
    t = np.linspace(0, 2 * np.pi, n_samples)
    x = 1.5 * np.cos(t)
    y = np.sin(t)
    z = np.sin(2 * t)
    X = np.vstack([np.column_stack([x, y, z]),
                   np.column_stack([-x, y - 1, -z])])
    labels = np.hstack([np.zeros(n_samples), np.ones(n_samples)])
    X += np.random.normal(scale=noise, size=X.shape)
    return X, labels

X_train, y_train = make_moons_3d(n_samples=500, noise=0.2)
X_test,  y_test  = make_moons_3d(n_samples=250, noise=0.2)

models = {
    'Decision Tree': DecisionTreeClassifier(max_depth=5),
    'AdaBoost + DT': AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=1), n_estimators=50),
    'SVM (Linear)':  SVC(kernel='linear'),
    'SVM (RBF)':     SVC(kernel='rbf'),
    'SVM (Poly)':    SVC(kernel='poly', degree=3)
}

results = []
for name, clf in models.items():
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    results.append({'Model': name, 'Accuracy': acc})

df_results = pd.DataFrame(results)
print(df_results)

# 3D visualization + confusion matrices (see main body for details)
```